

MIPS 汇编语言

程序数据段 (Data Segment) 定义整理

教材式学习讲义 | 定义、内存布局、访问方式与典型实例

适用范围	面向 MARS / QtSPIM 等常见 MIPS 教学环境；不同工具的默认地址或伪指令细节可能略有差异。
学习目标	能够读懂并编写 .data 中的变量、数组、字符串与缓冲区定义；正确使用地址、偏移与对齐。
阅读重点	把“标签是地址”这一核心概念，与 la / lw / sw 的分工连起来理解。

版本说明：本讲义以通用 MIPS 汇编知识为主，示例中的系统调用仅作场景说明；实际课程应以指定模拟器与教材约定为准。

阅读导航 建议学习路径

章节	核心问题	关键词
1. 基本概念	数据为何放在 .data ? 标签究竟代表什么 ?	data, label, address
2. 定义伪指令	不同类型占多少字节 , 如何初始化或预留空间 ?	.word, .byte, .space
3. 字符串与数组	连续内存怎样表达字符序列与下标访问 ?	.asciiz, offset, index
4. 访问数据	地址、内容、寄存器之间怎样转换 ?	la, lw, sw
5. 段与对齐	Data、Text、Stack 如何分工 ? 为什么需要 .align ?	.text, stack, alignment
6. 综合示例	如何把上述元素组织成可维护的程序数据定义 ?	constant, buffer, array

先记住一句话：数据段负责“放数据”；标签给出“数据的位置”；加载/存储指令负责“读写该位置”。

1 数据段的角色：程序中的静态数据

.data 是汇编器的段声明，表示后续内容是程序的静态数据，而不是可执行指令。它通常用于放置全局变量、静态变量、常量表、字符串、数组和预先申请的缓冲区。

程序装入内存后，这些数据已经拥有固定的位置。与运行时在栈上临时建立的局部变量不同，数据段中的对象通常在整个程序执行期间都可通过标签访问。

区域	典型内容	常见访问方式
.text	机器指令、过程代码	跳转或顺序取指
.data	已初始化数据、字符串、数组	la / lw / sw
stack	调用现场、局部变量、保存寄存器	以 \$sp 为基址的偏移访问

表 1 MIPS 中几个常见内存区域的功能对比

2 标签与地址：先区分“位置”和“内容”

标签 (label) 是汇编源代码中的符号名称。汇编器会把它绑定到当前位置；因此，当标签放在数据定义前面时，它可以理解为该数据对象的起始地址。标签不是寄存器，也不是数据内容本身。

例如 age: .word 18 中，age 对应一个地址；该地址处存放的 32 位字才是数值 18。

<pre>.data age: .word 18 .text la \$t0, age # \$t0 <- address of age lw \$t1, 0(\$t0) # \$t1 <- value stored at age (18)</pre>		
---	--	--

阅读数据访问指令时可连续问三个问题：基址寄存器里是哪个地址？偏移量是多少字节？该地址处按什么宽度读/写？这能避免把地址与内容混淆。

3 常用数据定义伪指令

伪指令	作用与大小	示例
.word	定义 32 位字；每个元素通常占 4 字节。	a: .word 10, 20
.half	定义 16 位半字；每个元素占 2 字节。	h: .half 1000
.byte	定义 8 位字节；适合字符、小范围整数或原始字节数据。	c: .byte 'A'
.space	预留指定字节数；其初值是否为零应以工具/环境规则为准。	buf: .space 128
.ascii	写入字符串字符本身，不自动附加结束字节。	s1: .ascii "OK"
.asciiz	写入字符串并自动追加 0 字节（C 风格结尾）。	s2: .asciiz "OK"
.align	将下一项起始地址对齐到 2 的某次幂边界（常见约定：.align 2 即 4 字节）。	.align 2
.eqv	定义汇编期符号常量；不分配运行时存储空间。	.eqv BUF_SIZE, 128

表 2 数据段常用伪指令。伪指令由汇编器处理，不等同于处理器执行的一条机器指令。

4 字符串：结束符决定能否按 C 字符串处理

字符串本质上是一串连续的字节。.ascii 只写入字符；.asciiz 会在最后自动加一个值为 0 的字节。许多教育模拟器的“打印字符串”服务需要的是后者，因为它会持续读取，直到遇到 0。

```
.data
raw:    .ascii  "ABC"          # bytes: 41 42 43
name:   .asciiz "ABC"          # bytes: 41 42 43 00

.text
la      $a0, name              # pass starting address
li      $v0, 4                 # MARS/QtSPIM print-string convention
syscall
```

注意：字符串中间也可以通过转义形式写入换行等控制字符。若要把字符串当作普通字节数组处理，仍应逐字节读取，并自行决定何时停止。

5 数组：连续元素与字节偏移

数组的元素在内存中连续排列。下标 i 的元素地址等于首地址加上 i × 每个元素字节数。因此，word 数组常用 4 倍下标，half 数组常用 2 倍下标，byte 数组直接使用下标。

```
.data
nums:    .word 3, 6, 9, 12

.text
la    $t0, nums        # base address
li    $t1, 2            # i = 2
sll   $t2, $t1, 2       # byte offset = i * 4
addu  $t3, $t0, $t2     # address of nums[i]
lw    $t4, 0($t3)       # $t4 = nums[2] = 9
```

对 byte 数组可用 lb/lbu 读取；对 half 数组可用 lh/luh；对 word 数组可用 lw。带 u 的版本执行零扩展，非 u 版本会按有符号数进行符号扩展。

6 访问数据：la、lw、sw 的分工

形式	作用	易混淆点
la \$t0, x	把标签 x 的地址装入寄存器。	得到的是位置，不是 x 的数值。
lw \$t1, x	教学汇编器常把它处理为从标签 x 对应位置读一个 word。	本质仍是“按地址取 4 字节”。
lw \$t1, 0(\$t0)	从 \$t0 + 0 处读 4 字节。	\$t0 必须是有效且适当对齐的地址。
sw \$t1, 4(\$t0)	向 \$t0 + 4 处写入一个 word。	偏移单位是字节，不是 word 个数。

表 3 地址装载与内存读写。实际可用语法可能受课程指定汇编器影响。

7 对齐：让多字节数据从合适边界开始

许多 MIPS 环境要求或强烈建议 word 地址是 4 的倍数、half 地址是 2 的倍数。`.align n` 通常会在当前位置填充若干字节，使下一项从 2^n 字节边界开始。例如，常见汇编器中 `.align 2` 用于让下一项从 4 字节边界开始。

```
.data
flag:    .byte 1
         .align 2           # align following word to 4-byte boundary
count:   .word 100
```

不要把 `.align` 解释成“申请 n 个字节”。它的目标是调整下一项的起始位置；实际填充量取决于当前位置。不同工具对参数含义的细节也可能不同，提交作业前应核对课程环境。

8 综合示例：一份可读的数据定义

下面示例同时使用了符号常量、字符串、标量、数组与可写缓冲区。注释强调每个标签的用途，能显著降低后续访问时写错长度或偏移的概率。

```
.eqv MAX_INPUT, 64

.data
prompt:      .asciiz "Enter an integer: "
newline:     .asciiz "\n"

        .align 2
count:       .word 0
values:      .word 5, 8, 13, 21

flags:       .byte 0, 1, 0, 1
input_buf:   .space MAX_INPUT

.text
main:
    la    $a0, prompt           # address of a null-terminated string
    li    $v0, 4
    syscall

    la    $t0, values
    lw    $t1, 4($t0)           # values[1] = 8
    sw    $t1, count           # count = 8
```

这里 `.eqv MAX_INPUT, 64` 只是把名称替换为常数；真正分配 64 字节的是 `input_buf: .space MAX_INPUT`。`values` 的元素是 `word`，因此偏移 4 对应第 2 个元素。

9 常见注意事项与自查清单

1. 地址 ≠ 内容：la 得到标签地址；lw/lb/lh 等才从地址读取内容。
2. 偏移按字节计：word 数组第 *i* 项的常见偏移是 $4*i$ ，不是 *i*。
3. 字符串输出通常需要 0 结尾：优先使用 `.asciiz`，除非明确需要原始字节序列。
4. `.space` 是字节数：`.space 40` 不是 40 个 word，而是 40 个字节。
5. 读写宽度要匹配：byte/half/word 选择相应的加载、存储指令，并留意符号扩展。
6. 对齐要主动核对：byte 后紧接 word 时，必要时使用 `.align`。
7. `.eqv` 不占用数据段空间：它是汇编期名称，不是运行时变量。
8. 模拟器约定有差异：默认数据地址、系统调用号、`.align` 细节应以课程工具说明为准。

复习提问：若看到 `sw $t1, 12($t0)`，请先回答“`$t0` 指向哪个对象？12 是多少字节偏移？写入对象应当是 byte、half 还是 word？”能够回答这三点，就已掌握大部分数据段访问问题。